SME: preparación y análisis de atributos en R

Ekhi Arzac

Contenido

1 Presentación y datos **1**
1.1 Subconjunto binario para el AUC . . . 2

2 Discretización **2**
2.1 Un atributo . . . 2
2.2 Dataset completo . . . 3
2.3 Cortes explícitos y funciones auxiliares . . . 3

3 Métricas de los atributos **4**
3.1 Estadísticos univariantes . . . 4
3.2 Reconocimiento del tipo en un dataset . . . 4
3.3 Entropía cruzada e información mutua . . . 5

4 Normalización y estandarización **5**

5 Filtrado por métricas **6**

6 Correlación y asociación por pares **6**

7 AUC y gráficos con ggplot2 **7**
7.1 Curva ROC . . . 7
7.2 Matrices de Pearson e información mutua . . . 9

8 Objeto S3 y lectura/escritura **9**

9 Referencia y reproducción de la entrega **11**
9.1 Consultar la viñeta instalada . . . 12
9.2 Compilar desde el proyecto . . . 12

1 Presentación y datos

Esta viñeta corresponde exclusivamente a la entrega de R. Muestra el uso de todas las funciones públicas de `sme`, con sus entradas, salidas e interpretación. Los algoritmos de discretización, métricas, transformación, selección y asociación son implementaciones propias. Se usan operaciones básicas de R y ggplot2 para dibujar.

Los ejemplos utilizan `datasets::iris`: medidas originales de flores recogidas por Anderson y utilizadas por Fisher. No se generan observaciones ni se añaden variables artificiales. La documentación de Iris describe los datos y sus referencias. Las medidas están en centímetros y `Species` identifica la especie.

```
iris_data <- datasets::iris
head(iris_data)
```

```
##   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## 1          5.1         3.5          1.4         0.2  setosa
## 2          4.9         3.0          1.4         0.2  setosa
## 3          4.7         3.2          1.3         0.2  setosa
```

```
## 4          4.6         3.1          1.5         0.2  setosa
## 5          5.0         3.6          1.4         0.2  setosa
## 6          5.4         3.9          1.7         0.4  setosa
```

```
dim(iris_data)
```

```
## [1] 150   5
```

```
table(iris_data$Species)
```

```
##
##     setosa versicolor  virginica
##         50         50         50
```

1.1 Subconjunto binario para el AUC

El AUC implementado necesita exactamente dos clases. Para los ejemplos supervisados se conservan todas las observaciones de **versicolor y virginica**, sin cambiar sus valores ni sus etiquetas. `virginica` es la clase positiva. Las otras operaciones pueden utilizar Iris completo.

```
iris_binary <- droplevels(subset(iris_data, Species != "setosa"))
positive <- "virginica"
table(iris_binary$Species)
```

```
##
## versicolor  virginica
##         50         50
```

Las numéricas se consideran continuas; factores, caracteres y lógicos se consideran categóricos. El argumento `categorical` permite declarar códigos numéricos como categorías cuando sea necesario. Los estadísticos omiten valores ausentes y las operaciones por pares usan pares completos. Las transformaciones conservan los NA. Iris no contiene ausentes y no se modifican los datos para introducirlos.

2 Discretización

`discretizeEW(x, num.bins, exclude)` divide el rango observado en intervalos de igual anchura. Para k intervalos, los cortes son $\min(x) + j(\max(x) - \min(x))/k$, con $j = 1, \ldots, k-1$. Es útil para conservar distancias en la escala original; no garantiza igual número de observaciones por intervalo.

`discretizeEF(x, num.bins, exclude)` ordena las observaciones y elige cortes próximos a las posiciones nj/k. Los valores iguales permanecen juntos: los empates pueden generar frecuencias desiguales y menos intervalos que los solicitados. Se eliminan cortes repetidos y el máximo observado.

Ambas reciben un vector numérico o un data frame y un entero positivo de intervalos. Devuelven etiquetas `I1`, `I2`, etc., o un nuevo data frame. Los métodos S3 `.numeric` y `.data.frame` se seleccionan automáticamente. Los límites pertenecen al intervalo inferior; las constantes generan `I1` y los NA se conservan.

2.1 Un atributo

```
x <- iris_data$Petal.Length
width <- discretizeEW(x, 3)
frequency <- discretizeEF(x, 3)
head(data.frame(Petal.Length = x, anchura = width, frecuencia = frequency), 8)
```

```
##   Petal.Length anchura frecuencia
## 1          1.4      I1         I1
## 2          1.4      I1         I1
## 3          1.3      I1         I1
## 4          1.5      I1         I1
## 5          1.4      I1         I1
## 6          1.7      I1         I1
```

```
## 7          1.4      I1         I1
## 8          1.5      I1         I1
```

```
rbind(anchura = table(width), frecuencia = table(frequency))
```

```
##            I1 I2 I3
## anchura    50 54 46
## frecuencia 50 54 46
```

Las tablas de frecuencias permiten apreciar la diferencia entre los algoritmos sobre las medidas reales. No se fuerza una separación entre observaciones empatadas.

2.2 Dataset completo

Solo se transforman las columnas numéricas no excluidas. La especie se conserva como factor; se indica también en `exclude` para expresar su papel en el análisis.

```
iris_ew <- discretizeEW(iris_data, 3, exclude = "Species")
iris_ef <- discretizeEF(iris_data, 3, exclude = "Species")
head(iris_ew)
```

```
##   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## 1           I1          I2           I1          I1  setosa
## 2           I1          I2           I1          I1  setosa
## 3           I1          I2           I1          I1  setosa
## 4           I1          I2           I1          I1  setosa
## 5           I1          I2           I1          I1  setosa
## 6           I1          I3           I1          I1  setosa
```

```
head(iris_ef)
```

```
##   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## 1           I1          I3           I1          I1  setosa
## 2           I1          I2           I1          I1  setosa
## 3           I1          I2           I1          I1  setosa
## 4           I1          I2           I1          I1  setosa
## 5           I1          I3           I1          I1  setosa
## 6           I1          I3           I1          I1  setosa
```

2.3 Cortes explícitos y funciones auxiliares

`discretize(x, cut.points, label.prefix)` permite elegir los cortes y devuelve un vector de etiquetas; ordena y deduplica los límites. Aquí se calculan a partir del rango observado del pétalo, sin introducir medidas nuevas.

`findRange(value, cut.points)` busca por búsqueda binaria el índice de intervalo, desde 1. Si se usa directamente, requiere cortes ordenados sin NA.

```
limits <- min(x) + (max(x) - min(x)) * (1:2) / 3
limits
```

```
## [1] 2.967 4.933
```

```
head(discretize(x, limits, label.prefix = "P"))
```

```
## [1] "P1" "P1" "P1" "P1" "P1" "P1"
```

```
findRange(x[1], limits)
```

```
## [1] 1
```

`applyColumns(df, filter, fn)` aplica una transformación a las columnas seleccionadas por un predicado y devuelve el data frame transformado. Este ejemplo reutiliza la normalización propia sobre las medidas, conservando la especie.

```
head(applyColumns(iris_data, is.numeric, normalize))
```

```
##   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## 1      0.22222      0.6250      0.06780     0.04167  setosa
## 2      0.16667      0.4167      0.06780     0.04167  setosa
## 3      0.11111      0.5000      0.05085     0.04167  setosa
## 4      0.08333      0.4583      0.08475     0.04167  setosa
## 5      0.19444      0.6667      0.06780     0.04167  setosa
## 6      0.30556      0.7917      0.11864     0.12500  setosa
```

3 Métricas de los atributos

3.1 Estadísticos univariantes

`mean(x)` calcula la media aritmética; `variance(x)` utiliza la varianza **muestral** $\sum_i (x_i - \bar{x})^2/(n-1)$, y `stdDev(x)` es su raíz cuadrada. Reciben un vector numérico y devuelven un escalar. Con menos de dos observaciones, la varianza y la desviación son indefinidas (`NA`).

La función propia `mean` conserva el nombre de la implementación del proyecto y oculta la genérica de R al cargar el paquete. Para solicitar expresamente la función original se puede usar `base::mean`.

`entropy(x)` recibe categorías y calcula la entropía de Shannon en bits: $H(X) = -\sum_x p(x) \log_2 p(x)$. Es cero para una constante. También puede aplicarse a las etiquetas producidas explícitamente por la discretización.

```
c(media = mean(x), varianza = variance(x), desviacion = stdDev(x))
```

```
##      media   varianza desviacion
##      3.758      3.116      1.765
```

```
entropy(iris_data$Species)
```

```
## [1] 1.585
```

```
entropy(iris_ef$Petal.Length)
```

```
## [1] 1.582
```

3.2 Reconocimiento del tipo en un dataset

`attributeMetrics(df, class, positive, categorical)` devuelve una fila por atributo: nombre, tipo, número observado, varianza, entropía y AUC. En numéricas calcula varianza y, si se especifica clase binaria, AUC; en categóricas, entropía. Excluye la clase de la evaluación de atributos. Lo no aplicable o indefinido se representa con `NA`. Sin clase, las tres especies pueden analizarse como un atributo categórico.

```
attributeMetrics(iris_data)
```

```
##      attribute        type   n variance entropy auc
## 1 Sepal.Length     numeric 150   0.6857      NA  NA
## 2  Sepal.Width     numeric 150   0.1900      NA  NA
## 3 Petal.Length     numeric 150   3.1163      NA  NA
## 4  Petal.Width     numeric 150   0.5810      NA  NA
## 5      Species categorical 150       NA   1.585  NA
```

```
metrics <- attributeMetrics(iris_binary, class = "Species", positive = positive)
metrics
```

```
##      attribute    type   n variance entropy    auc
## 1 Sepal.Length numeric 100   0.4393      NA 0.7896
## 2  Sepal.Width numeric 100   0.1107      NA 0.6636
## 3 Petal.Length numeric 100   0.6816      NA 0.9822
## 4  Petal.Width numeric 100   0.1804      NA 0.9804
```

El AUC considera la dirección original del atributo: una medida mayor predice la clase positiva. Un valor bajo puede corresponder a una relación inversa, que no se invierte automáticamente. Si tras omitir pares incompletos no quedan ambas clases, el AUC del atributo es `NA`.

3.3 Entropía cruzada e información mutua

`crossEntropy(x, y)` compara dos distribuciones categóricas empíricas mediante $H(P,Q) = -\sum_x p(x)\log_2 q(x)$. Alinea por etiqueta, admite muestras de diferente longitud y devuelve `Inf` si falta en Q una categoría observada en P. Se conserva de la implementación inicial. Comparar una distribución consigo misma devuelve su entropía.

`mutualInfo(x, y)` recibe dos vectores categóricos emparejados y calcula $\sum_{x,y} p(x,y)\log_2[p(x,y)/(p(x)p(y))]$ usando frecuencias conjuntas. No utiliza entropía cruzada. Su unidad es el bit y no está limitada a uno. Una variable consigo misma tiene información mutua igual a su entropía.

```
crossEntropy(iris_data$Species, iris_data$Species)
```

```
## [1] 1.585
```

```
mutualInfo(iris_data$Species, iris_data$Species)
```

```
## [1] 1.585
```

```
mutualInfo(iris_ef$Petal.Length, iris_data$Species)
```

```
## [1] 1.325
```

4 Normalización y estandarización

`normalize(x, exclude)` aplica $(x-\min(x))/(\max(x)-\min(x))$ y lleva el rango a $[0,1]$. `standardize(x, exclude)` aplica $(x-\bar{x})/s$ con desviación muestral. Esta última centra y ajusta la escala sin imponer un rango $[0,1]$.

Aceptan vector o data frame y devuelven un nuevo objeto, sin modificar Iris. En datasets solo se transforman las numéricas no excluidas. Las constantes generan ceros; al estandarizar también lo hace una columna con una sola observación. Se conservan NA y columnas no numéricas.

```
head(data.frame(original = x, normalizada = normalize(x),
                estandarizada = standardize(x)))
```

```
##   original normalizada estandarizada
## 1      1.4     0.06780        -1.336
## 2      1.4     0.06780        -1.336
## 3      1.3     0.05085        -1.392
## 4      1.5     0.08475        -1.279
## 5      1.4     0.06780        -1.336
## 6      1.7     0.11864        -1.166
```

```
normalized <- normalize(iris_data, exclude = "Species")
standardized <- standardize(iris_data, exclude = "Species")
head(normalized)
```

```
##   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## 1      0.22222      0.6250      0.06780     0.04167  setosa
## 2      0.16667      0.4167      0.06780     0.04167  setosa
## 3      0.11111      0.5000      0.05085     0.04167  setosa
## 4      0.08333      0.4583      0.08475     0.04167  setosa
## 5      0.19444      0.6667      0.06780     0.04167  setosa
## 6      0.30556      0.7917      0.11864     0.12500  setosa
```

```
head(standardized)
```

```
##   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## 1      -0.8977     1.01560       -1.336      -1.311  setosa
## 2      -1.1392    -0.13154       -1.336      -1.311  setosa
## 3      -1.3807     0.32732       -1.392      -1.311  setosa
## 4      -1.5015     0.09789       -1.279      -1.311  setosa
## 5      -1.0184     1.24503       -1.336      -1.311  setosa
## 6      -0.5354     1.93331       -1.166      -1.049  setosa
```

```
c(media = mean(standardized$Petal.Length),
  desviacion = stdDev(standardized$Petal.Length))
```

```
##      media desviacion
##  -2.75e-16   1.00e+00
```

5 Filtrado por métricas

`filterFeatures` recibe el dataset y umbrales mínimos inclusivos: se acepta un atributo si cumple todas sus condiciones aplicables. `min.variance` y `min.auc` se aplican a numéricas; `min.entropy`, a categóricas. Omitir un umbral no impone esa condición. Una métrica indefinida no supera un umbral solicitado.

La salida conserva el orden de las columnas seleccionadas y siempre incluye la clase indicada. Filtrar por AUC exige clase binaria. Los umbrales 0.1, 0.8 y 1.5 son **criterios ilustrativos**, no resultados asumidos ni parámetros aprendidos.

```
selected <- filterFeatures(iris_binary, min.variance = 0.1, min.auc = 0.8,
                           class = "Species", positive = positive)
names(selected)
```

```
## [1] "Petal.Length" "Petal.Width"  "Species"
```

```
head(selected)
```

```
##    Petal.Length Petal.Width    Species
## 51          4.7         1.4 versicolor
## 52          4.5         1.5 versicolor
## 53          4.9         1.5 versicolor
## 54          4.0         1.3 versicolor
## 55          4.6         1.5 versicolor
## 56          4.5         1.3 versicolor
```

```
# Ejemplo de filtrado categórico sobre las medidas discretizadas de Iris.
selected_discrete <- filterFeatures(iris_ef, min.entropy = 1.5)
names(selected_discrete)
```

```
## [1] "Sepal.Length" "Sepal.Width"  "Petal.Length" "Petal.Width"  "Species"
```

```
head(selected_discrete)
```

```
##   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## 1           I1          I3           I1          I1  setosa
## 2           I1          I2           I1          I1  setosa
## 3           I1          I2           I1          I1  setosa
## 4           I1          I2           I1          I1  setosa
## 5           I1          I3           I1          I1  setosa
## 6           I1          I3           I1          I1  setosa
```

6 Correlación y asociación por pares

`pearson(x, y)` recibe vectores numéricos emparejados y devuelve un coeficiente en $[-1, 1]$. Calcula productos de desviaciones respecto a las medias. El signo indica la dirección de una relación lineal. Con constantes o menos de dos pares completos es indefinido.

`associationMatrix(df, categorical, num.bins)` devuelve una lista con matrices simétricas `values` y `methods`. Dos numéricas usan Pearson; dos categóricas, información mutua. En pares mixtos se discretiza la numérica por igual frecuencia sobre los pares completos y se calcula MI (`mi_discretized`). Este resultado depende del número de intervalos, que queda indicado en la llamada.

```
pearson(iris_data$Petal.Length, iris_data$Petal.Width)
```

```
## [1] 0.9629
```

```
associations <- associationMatrix(iris_data, num.bins = 3)
round(associations$values, 3)
```

```
##              Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## Sepal.Length        1.000      -0.118        0.872       0.818   0.647
## Sepal.Width        -0.118       1.000       -0.428      -0.366   0.357
## Petal.Length        0.872      -0.428        1.000       0.963   1.325
## Petal.Width         0.818      -0.366        0.963       1.000   1.369
## Species             0.647       0.357        1.325       1.369   1.585
```

```
associations$methods
```

```
##              Sepal.Length     Sepal.Width      Petal.Length     Petal.Width
## Sepal.Length "pearson"        "pearson"        "pearson"        "pearson"
## Sepal.Width  "pearson"        "pearson"        "pearson"        "pearson"
## Petal.Length "pearson"        "pearson"        "pearson"        "pearson"
## Petal.Width  "pearson"        "pearson"        "pearson"        "pearson"
## Species      "mi_discretized" "mi_discretized" "mi_discretized" "mi_discretized"
##              Species
## Sepal.Length "mi_discretized"
## Sepal.Width  "mi_discretized"
## Petal.Length "mi_discretized"
## Petal.Width  "mi_discretized"
## Species      "mi"
```

```
# Tras discretizar todas las medidas, todos los pares utilizan MI.
discrete_associations <- associationMatrix(iris_ef)
round(discrete_associations$values, 3)
```

```
##              Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## Sepal.Length        1.575       0.240        0.712       0.660   0.647
## Sepal.Width         0.240       1.576        0.368       0.378   0.357
## Petal.Length        0.712       0.368        1.582       1.232   1.325
## Petal.Width         0.660       0.378        1.232       1.584   1.369
## Species             0.647       0.357        1.325       1.369   1.585
```

7 AUC y gráficos con ggplot2

7.1 Curva ROC

`rocCurve(scores, class, positive)` devuelve la tabla de umbrales, tasa de falsos positivos y tasa de verdaderos positivos. Comienza en $(0, 0)$ y termina en $(1, 1)$; los scores empatados se incorporan en un mismo punto. `auc` integra trapecios entre los puntos: un empate entre observaciones de distinta clase aporta media victoria.

Aquí los scores son las medidas originales del pétalo del subconjunto binario. No se entrena un modelo ni se inventan puntuaciones. `plotROC` dibuja con ggplot2 y devuelve invisiblemente `curve`, `auc` y `plot`, un objeto ggplot reutilizable.

```
scores <- iris_binary$Petal.Length
classes <- iris_binary$Species
auc(scores, classes, positive = positive)
```

```
## [1] 0.9822
```

```
roc <- rocCurve(scores, classes, positive = positive)
head(roc)
```

```
##   threshold fpr  tpr
## 1       Inf   0 0.00
## 2       6.9   0 0.02
## 3       6.7   0 0.06
```

```
## 4       6.6   0 0.08
## 5       6.4   0 0.10
## 6       6.3   0 0.12
```

```
tail(roc)
```

```
##    threshold  fpr tpr
## 30       3.8 0.86   1
## 31       3.7 0.88   1
## 32       3.6 0.90   1
## 33       3.5 0.94   1
## 34       3.3 0.98   1
## 35       3.0 1.00   1
```

```
roc_plot <- plotROC(scores, classes, positive = positive,
                    main = "Iris: virginica frente a versicolor")
```

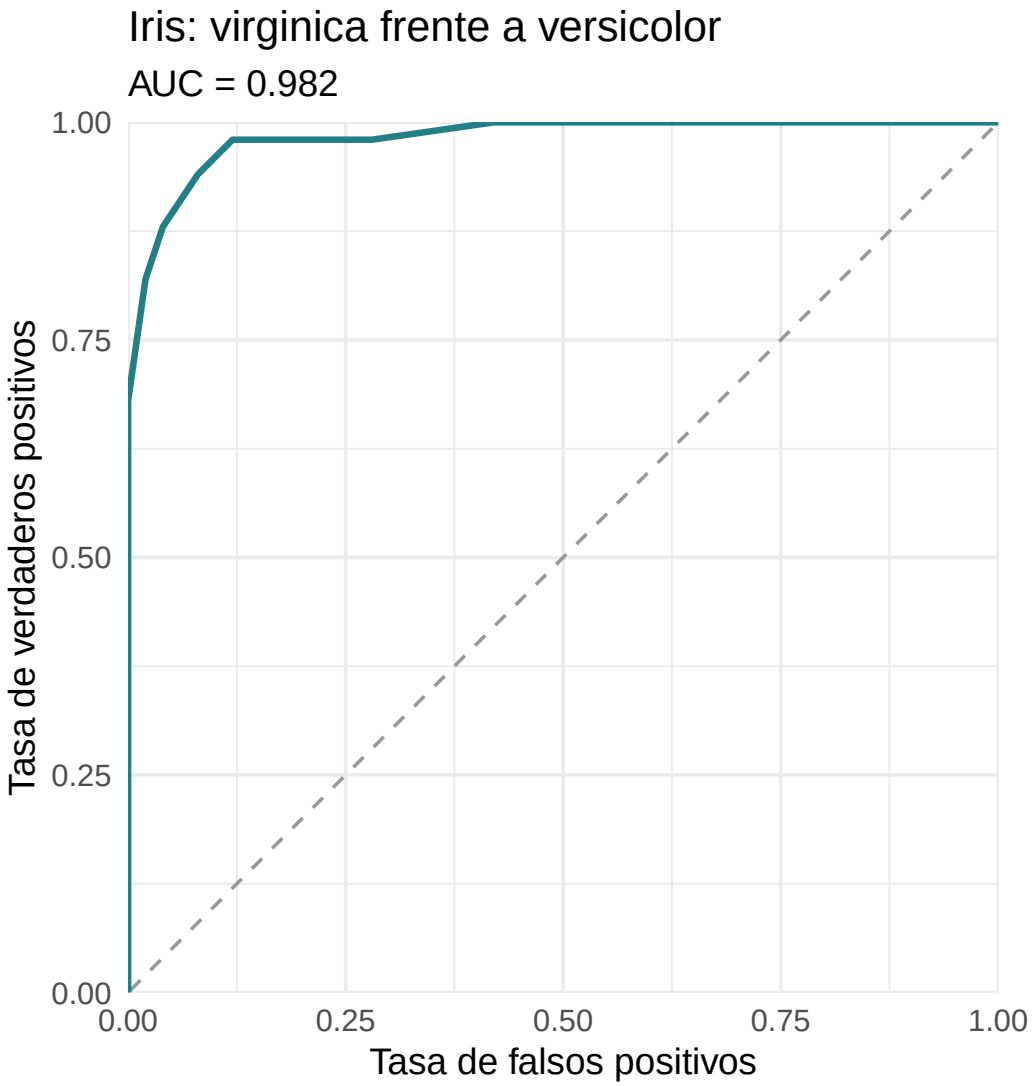


Figura 1: ROC de la longitud del pétalo para distinguir virginica de versicolor.

El área representa la capacidad de ordenar observaciones positivas por encima de negativas; la diagonal corresponde a un AUC de 0.5. La función respeta la dirección original de la medida.

7.2 Matrices de Pearson e información mutua

`plotAssociation` recibe la lista de asociaciones y dibuja dos paneles. Pearson utiliza la escala $[-1, 1]$; MI tiene su propia escala en bits. El gris indica una celda ausente o correspondiente al otro método. Cero en MI es un valor válido. En la diagonal categórica aparece la entropía, no necesariamente uno.

```
association_plots <- plotAssociation(associations, main = "Iris")
```



Figura 2: Asociaciones de Iris: escalas independientes para Pearson y MI.

La salida conserva `values` y `methods` y añade `plots$pearson` y `plots$mi`. Los gráficos pueden personalizarse o guardarse con `ggplot2::ggsave`. Este ejemplo guarda la curva ya calculada en un archivo temporal, sin alterar los datos.

```
plot_file <- tempfile(fileext = ".pdf")
ggplot2::ggsave(plot_file, plot = roc_plot$plot, width = 6, height = 5)
file.exists(plot_file)
```

```
## [1] TRUE
```

```
unlink(plot_file)
```

8 Objeto S3 y lectura/escritura

`smeDataset(data, class, positive, categorical)` reúne los datos y sus metadatos. Los genéricos anteriores seleccionan métodos `.sme_dataset`; las transformaciones protegen automáticamente la clase y las categorías declaradas y devuelven otro objeto. `attributeMetrics`, `filterFeatures` y `associationMatrix` reutilizan los metadatos, evitando repetirlos en cada llamada. El filtrado actualiza los nombres categóricos si se elimina alguna columna.

```
ds <- smeDataset(iris_binary, class = "Species", positive = positive)
class(ds)
```

```
## [1] "sme_dataset"
```

```
head(as.data.frame(ds))
```

```
##    Sepal.Length Sepal.Width Petal.Length Petal.Width    Species
## 51          7.0         3.2          4.7         1.4 versicolor
## 52          6.4         3.2          4.5         1.5 versicolor
## 53          6.9         3.1          4.9         1.5 versicolor
## 54          5.5         2.3          4.0         1.3 versicolor
## 55          6.5         2.8          4.6         1.5 versicolor
## 56          5.7         2.8          4.5         1.3 versicolor
```

```
attributeMetrics(ds)
```

```
##      attribute    type   n variance entropy    auc
## 1 Sepal.Length numeric 100   0.4393      NA 0.7896
## 2  Sepal.Width numeric 100   0.1107      NA 0.6636
## 3 Petal.Length numeric 100   0.6816      NA 0.9822
## 4  Petal.Width numeric 100   0.1804      NA 0.9804
```

```
head(as.data.frame(normalize(ds)))
```

```
##    Sepal.Length Sepal.Width Petal.Length Petal.Width    Species
## 51       0.7000      0.6667       0.4359      0.2667 versicolor
## 52       0.5000      0.6667       0.3846      0.3333 versicolor
## 53       0.6667      0.6111       0.4872      0.3333 versicolor
## 54       0.2000      0.1667       0.2564      0.2000 versicolor
## 55       0.5333      0.4444       0.4103      0.3333 versicolor
## 56       0.2667      0.4444       0.3846      0.2000 versicolor
```

```
head(as.data.frame(standardize(ds)))
```

```
##    Sepal.Length Sepal.Width Petal.Length Petal.Width    Species
## 51       1.1134      0.9857    -0.249522     -0.6498 versicolor
## 52       0.2082      0.9857    -0.491776     -0.4143 versicolor
## 53       0.9625      0.6852    -0.007268     -0.4143 versicolor
## 54      -1.1496     -1.7190    -1.097412     -0.8852 versicolor
## 55       0.3591     -0.2164    -0.370649     -0.4143 versicolor
## 56      -0.8479     -0.2164    -0.491776     -0.8852 versicolor
```

```
head(as.data.frame(discretizeEW(ds, 3)))
```

```
##    Sepal.Length Sepal.Width Petal.Length Petal.Width    Species
## 51           I3          I2           I2          I1 versicolor
## 52           I2          I2           I2          I1 versicolor
## 53           I2          I2           I2          I1 versicolor
## 54           I1          I1           I1          I1 versicolor
## 55           I2          I2           I2          I1 versicolor
## 56           I1          I2           I2          I1 versicolor
```

```
head(as.data.frame(discretizeEF(ds, 3)))
```

```
##    Sepal.Length Sepal.Width Petal.Length Petal.Width    Species
## 51           I3          I3           I2          I1 versicolor
## 52           I2          I3           I1          I2 versicolor
## 53           I3          I3           I2          I2 versicolor
## 54           I1          I1           I1          I1 versicolor
## 55           I2          I1           I2          I2 versicolor
## 56           I1          I1           I1          I1 versicolor
```

```
names(as.data.frame(filterFeatures(ds, min.auc = 0.8)))
```

```
## [1] "Petal.Length" "Petal.Width"  "Species"
```

```
round(associationMatrix(ds, num.bins = 3)$values, 3)
```

```
##              Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## Sepal.Length        1.000       0.554        0.828       0.594   0.138
## Sepal.Width         0.554       1.000        0.520       0.566   0.034
## Petal.Length        0.828       0.520        1.000       0.823   0.644
## Petal.Width         0.594       0.566        0.823       1.000   0.634
## Species             0.138       0.034        0.644       0.634   1.000
```

`print(ds)` muestra las dimensiones, la clase y sus datos. Se ejecuta aquí sin volcar las cien filas al documento.

```
print(ds)
```

`writeDataset(x, file)` guarda valores en CSV sin nombres de fila y devuelve invisiblemente la ruta. `readDataset(file, ...)` lee el CSV y construye el objeto. CSV no conserva metadatos ni todos los tipos; se especifican de nuevo al leer y `colClasses` permite controlar la inferencia. El archivo temporal contiene solo observaciones de Iris.

```
path <- tempfile(fileext = ".csv")
writeDataset(ds, path)
loaded <- readDataset(path, class = "Species", positive = positive)
head(as.data.frame(loaded))
```

```
##   Sepal.Length Sepal.Width Petal.Length Petal.Width    Species
## 1          7.0         3.2          4.7         1.4 versicolor
## 2          6.4         3.2          4.5         1.5 versicolor
## 3          6.9         3.1          4.9         1.5 versicolor
## 4          5.5         2.3          4.0         1.3 versicolor
## 5          6.5         2.8          4.6         1.5 versicolor
## 6          5.7         2.8          4.5         1.3 versicolor
```

```
attributeMetrics(loaded)
```

```
##      attribute    type   n variance entropy    auc
## 1 Sepal.Length numeric 100   0.4393      NA 0.7896
## 2  Sepal.Width numeric 100   0.1107      NA 0.6636
## 3 Petal.Length numeric 100   0.6816      NA 0.9822
## 4  Petal.Width numeric 100   0.1804      NA 0.9804
```

```
unlink(path)
```

9 Referencia y reproducción de la entrega

Las funciones y métodos S3 mostrados están documentados también en la ayuda del paquete. Las funciones auxiliares internas de validación no forman parte de la interfaz pública.

Función o genérico	Entrada principal	Salida
`discretizeEW`, `discretizeEF`	vector/dataset, intervalos	etiquetas o dataset
`discretize`	vector y cortes	etiquetas
`findRange`	valor y cortes ordenados	índice desde 1
`applyColumns`	datos, predicado y función	data frame
`mean`, `variance`, `stdDev`	vector numérico	escalar
`entropy`, `crossEntropy`, `mutualInfo`	categorías	bits
`attributeMetrics`	datos, clase y tipos	tabla de métricas
`normalize`, `standardize`	vector/dataset	datos transformados
`filterFeatures`	datos y umbrales	datos seleccionados
`pearson`	vectores numéricos	coeficiente
`associationMatrix`	datos, tipos e intervalos	valores y métodos
`auc`, `rocCurve`	scores y clase binaria	área o tabla

Función o genérico	Entrada principal	Salida
`plotROC`, `plotAssociation`	scores/asociaciones	gráficos ggplot
`smeDataset`	datos y metadatos	objeto S3
`print`, `as.data.frame`	objeto S3	presentación o datos
`readDataset`, `writeDataset`	objeto/ruta CSV	objeto o archivo
`openGuide`	abrir o localizar	ruta del PDF

9.1 Consultar la viñeta instalada

Las llamadas siguientes se utilizan después de construir e instalar el paquete con su PDF. No se ejecutan durante la compilación del propio documento.

```
vignette("entrega", package = "sme")
openGuide(browse = FALSE)  # Ruta del PDF instalado.
openGuide(browse = TRUE)   # Abrir el PDF en el visor.
```

9.2 Compilar desde el proyecto

Desde `SME-r`, con el paquete instalado y una distribución LaTeX que incluya XeLaTeX:

```
install.packages(c("ggplot2", "knitr", "rmarkdown"))
rmarkdown::render("vignettes/entrega.Rmd", output_dir = "inst/doc")
```

El fuente es `vignettes/entrega.Rmd` y el resultado es `inst/doc/entrega.pdf`. `R CMD build SME-r` construye el paquete y su viñeta mediante knitr/rmarkdown. Para instalar desde el repositorio con reconstrucción de viñetas:

```
devtools::install_github("ekhi-arzac/SME-r", build_vignettes = TRUE)
```